

## 33.1 Counting Sort

Imagine if instead of driving a slow Honda Civic, we started driving a fast Ferrari. Unfortunately, we won't actually be driving in a Ferrari today, but we will witness a blazing fast algorithm that's just as fast called Radix Sorts.

When sorting an array, sorting requires  $\mathcal{O}(N \log N)$  compare operations in the worst case (array is sorted in descending order). Thus, the ultimate comparison based sorting algorithm has a worst case runtime of  $\mathcal{O}(N \log N)$ .

From an asymptotic perspective, that means no matter how clever we are, we can never beat Merge Sort's worst case runtime of  $\mathcal{O}(N \log N)$ . But what if we don't compare at all?

| #  |         |              |                     |
|----|---------|--------------|---------------------|
| 5  | Sandra  | Vanilla      | Grimes              |
| 0  | Lauren  | Mint         | Jon Talabot         |
| 11 | Lisa    | Vanilla      | Blue Peter          |
| 9  | Dave    | Chocolate    | Superpope           |
| 4  | JS      | Fish         | The Filthy Reds     |
| 7  | James   | Rocky Road   | Robots are Supreme  |
| 3  | Edith   | Vanilla      | My Bloody Valentine |
| 6  | Swimp   | Chocolate    | Sef                 |
| 1  | Delbert | Strawberry   | Ronald Jenkees      |
| 2  | Glaser  | Cardamom     | Rx Nightly          |
| 8  | Lee     | Vanilla      | La(r)va             |
| 10 | Bearman | Butter Pecan | Extrobophile        |

| #  |         |       |
|----|---------|-------|
| 0  | Lauren  |       |
| 1  | Delbert | Str   |
| 2  | Glaser  | Car   |
| 3  | Edith   | V     |
| 4  | JS      |       |
| 5  | Sandra  | V     |
| 6  | Swimp   | Ch    |
| 7  | James   | Roc   |
| 8  | Lee     | V     |
| 9  | Dave    | Ch    |
| 10 | Bearman | Butte |
| 11 | Lisa    | V     |

Left is original, right is ordered output

Essentially what just happened is that we first made a new array of the same size and then just copied all of the # indexes to the correct location. So first we look at 5 Sandra Vanilla Grimes and then copy this over to the 5th index in our new array.

This does guarantee  $\Theta(N)$  worst case time. However what if we were working with

- Non-unique keys.

- Non-consecutive keys.
- Non-numerical keys.

All of these cases are complex cases that aren't so simple to deal with. Essentially what we can do is create a simpler method which is to:

- Count number of occurrences of each item.
- Iterate through list, using count array to decide where to put everything.

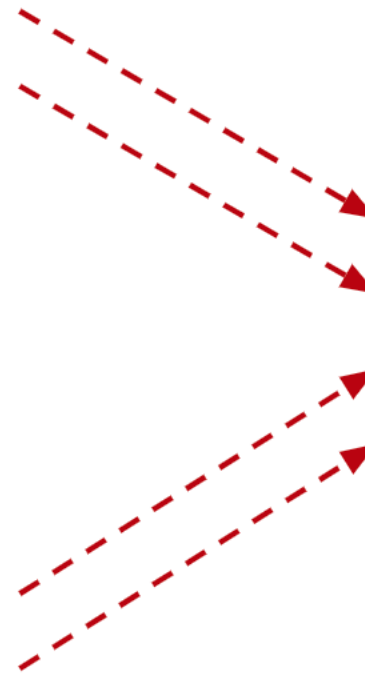
Bottom line, we can use counting sort to sort  $N$  objects in  $\Theta(N)$  time.

## 33.2 LSD Radix Sort

| 22 | <b>Lauren</b> |
|----|---------------|
| 34 | Delbert       |
| 41 | Glaser        |
| 13 | Edith         |
| 23 | JS            |
| 41 | Sandra        |
| 32 | <b>Swimp</b>  |
| 34 | James         |
| 12 | Lee           |
| 31 | Dave          |
| 12 | Bearman       |
| 42 | Lisa          |



| 41 | Glaser  |
|----|---------|
| 41 | Sandra  |
| 31 | Dave    |
| 32 | Swimp   |
| 22 | Lauren  |
| 12 | Lee     |
| 12 | Bearman |
| 42 | Lisa    |
| 23 | JS      |
| 13 | Edith   |
| 34 | James   |
| 34 | Delbert |



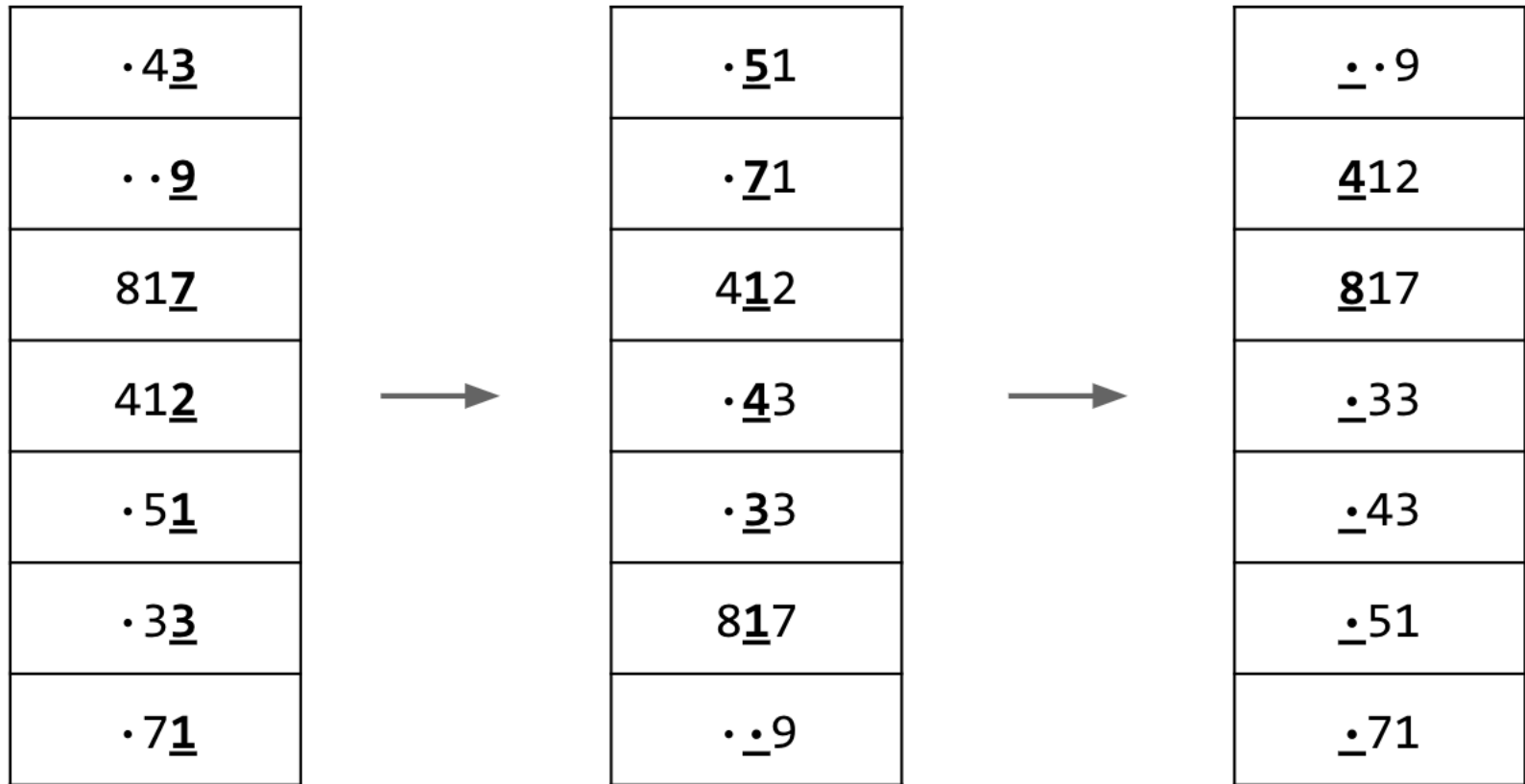
Least Significant Digit Radix Sort – Using Count Sort

Notice in the above picture that we had a completely random input of numbers 22, 34, 41, etc. Then in the second box, we had sorted by right most digit, so 41, 41, 31 would come first as 1 is the lowest rightmost digit. Next would come 32, 22, 12, as 2 is the second least rightmost digit. Notice that we still have to take care of sorting 32, 22, and 12 which is why we move to the second right most digit in our last box which has everything sorted.

What is the runtime of LSD sort?

- $\Theta(WN + WR)$
- $N$ : Number of items,  $R$ : size of alphabet,  $W$ : Width of each item in # digits

When keys are of different lengths, can treat empty spaces as characters.

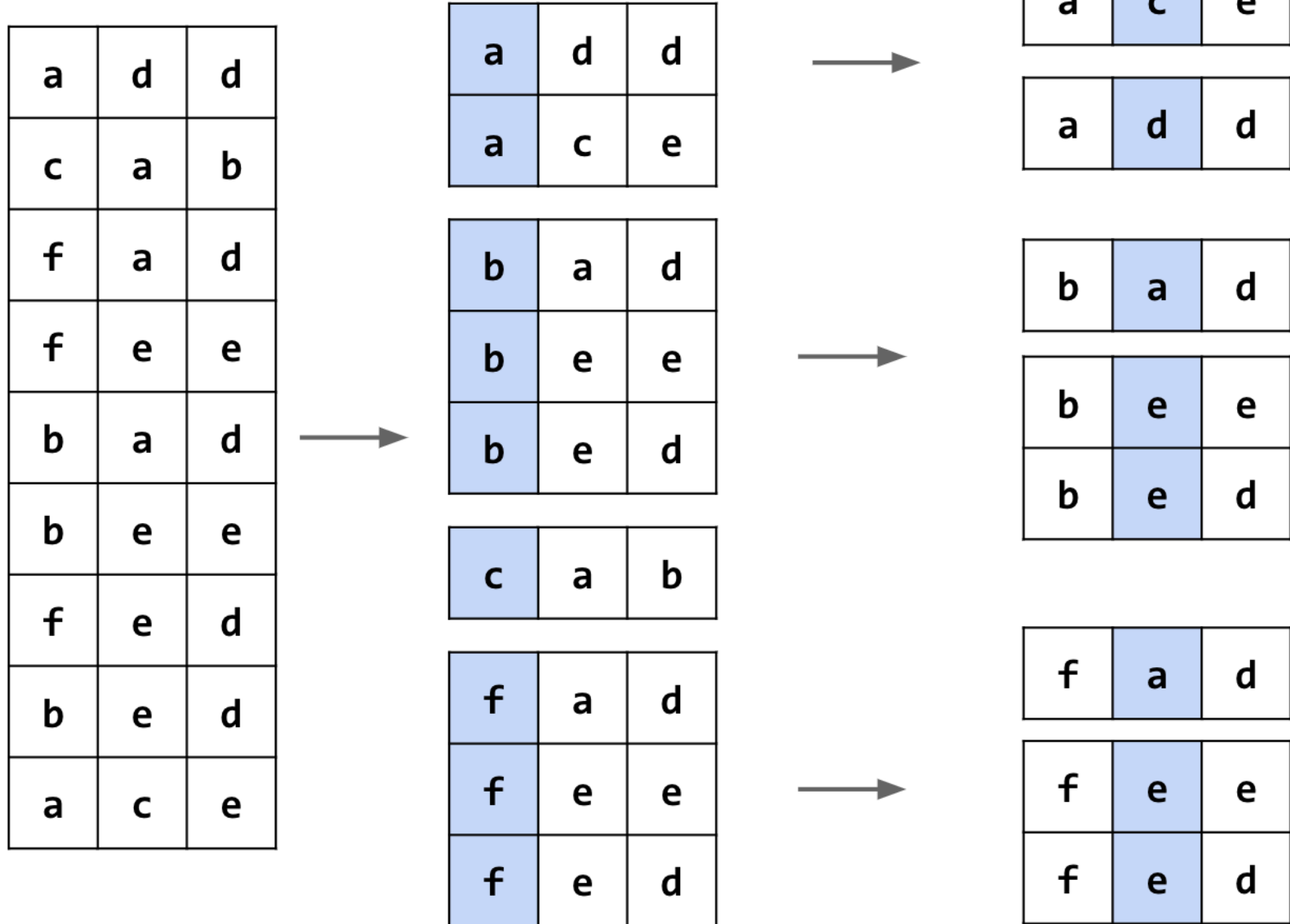


Non-equal Key Lengths

### 33.3 MSD Radix Sort

- Suppose we sort by topmost digit, then middle digit, then rightmost digit. Will we arrive at the correct result?

Key idea: Sort each subproblem separately.



MSD Radix Sort (correct edition)

Notice first we sorted by leftmost digit. Then we grouped the data by the leftmost digit, so one group would start with a's, then the next group would start with b's, and so on and so forth. Then within our subgroups we would order by middle digit, and create newer subgroups. And finally we would break this up into further subgroups until we have all individual subproblems. This final result would be sorted.

**Runtimes**

Best Case.

- We finish in one counting sort pass, looking only at the top digit:  $\Theta(N + R)$

Worst Case.

- We have to look at every character, degenerating to LSD sort:  $\Theta(WN + WR)$

**Summary of Runtimes**

|                  | Memory           | Runtime (worst)                                 | Notes                                      |
|------------------|------------------|-------------------------------------------------|--------------------------------------------|
| Heapsort         | $\Theta(1)$      | $\Theta(N \log N)^*$                            | Bad caching (61C)                          |
| Insertion        | $\Theta(1)$      | $\Theta(N^2)^*$                                 | Fastest for small N,<br>almost sorted data |
| Mergesort        | $\Theta(N)$      | $\Theta(N \log N)^*$                            | Fastest stable sort                        |
| Random Quicksort | $\Theta(\log N)$ | $\Theta(N \log N)^*$ expected                   | Fastest compare sort                       |
| Counting Sort    | $\Theta(N+R)$    | $\Theta(N+R)$                                   | Alphabet keys only                         |
| LSD Sort         | $\Theta(N+R)$    | $\Theta(WN+WR)$                                 | Strings of alphabetical<br>keys only       |
| MSD Sort         | $\Theta(N+WR)$   | $\Theta(N+R)$ (best)<br>$\Theta(WN+WR)$ (worst) | Bad caching (61C)                          |

## 33.4 Summary

### Terminology.

- Radix - just another word for ‘base’ as in the base of a number system. For example, the radix for words written in lowercase English letters is 26. For number written in Arabic numerals it is 10.
- Radix sort - a sort that works with one character at a time (by grouping objects that have the same digit in the same position).
- Note: I will use ‘character’ and ‘digit’ interchangeably in this study guide.

**Counting Sort.** Allows you to sort  $N$  keys that are integers between 0 and  $R - 1$  in  $\Theta(N + R)$  time. Beats linearithmic lower bound by avoiding any binary compares. This is a completely different philosophy for how things should be sorted. This is the most important concept for this lecture.

**LSD.** In the LSD algorithm, we sort by each digit, working from right to left. Requires examination of  $\Theta(WN)$  digits, where  $W$  is the length of the longest key. Runtime is  $\Theta(WN + WR)$ , though we usually think of  $R$  as a constant and just say  $\Theta(WN)$ . The  $\Theta(WR)$  part of the runtime is due to the creation of length  $R$  arrows for counting sort. We usually do LSD sort using counting sort as a subroutine, but it’s worth thinking about whether other sorts might work as well.

**LSD vs Comparison Sorting.** Our comparison sorts, despite requiring  $\Theta(N * \log N)$  time, can still be faster than LSD sort. For extremely large  $N$ , LSD sort will naturally win, but  $\log N$  is typically pretty small. Know which algorithm is best in the two extreme cases of very long dissimilar strings and very long, nearly equal strings.

**MSD.** In MSD sorting, we work from left to right, and solve each resulting subproblem independently. Thus, for each problem, we may have as many as  $R$  subproblem. Worst case runtime is exactly the same as LSD sort,  $\Theta(WN + WR)$ , though can be much better. In the very best case, where we only have to look at the top character (only possible for  $R > N$ ), we have a runtime of  $\Theta(N + R)$ .

## 33.5 Exercises

### Factual

1. If we use counting sort to sort all 61B students by the number of siblings they have, how long will the counting array be? Let  $N$  be the total number of students and  $M$  be the maximum number of siblings a student has.
2. When performing a radix sort like LSD or MSD, which of the following sorting algorithms could we use for each digit?
  - ☐ Counting sort
  - ☐ Heapsort
  - ☐ Mergesort
  - ☐ Selection sort
3. If we’re sorting  $N$  lowercase english strings using MSD sort, what is the maximum number of subproblems after considering only the the most significant digit?

- Problem 1
- Problem 2
- Problem 3

### Metacognitive

1. What are some advantages of counting sort over quicksort?
2. Why do we use LSD over counting sort?
3. If we use MSD radix sort, we start with a single problem of size  $N$ , where  $N$  is the number of strings. Depending on the results on the most significant digit, we end up with a larger number of smaller subproblems, i.e. we’re doing some divide and conquer. In the worst case, we end up with only one subproblem, also of size  $N$ . When does this happen?

- Problem 1
- Problem 2
- Problem 3

## 33. Radix Sorts